

Módulo 4 – Capítulo 08 – Sección 01

Escalabilidad en Plataformas de IA

Escalar un sistema de IA es considerablemente más complejo que escalar un sistema de software convencional. En una aplicación web tradicional, escalar horizontalmente significa agregar más instancias del servidor de aplicaciones detrás de un balanceador de carga. En un sistema de IA, la demanda se distribuye entre componentes con perfiles de carga radicalmente distintos: el servicio de inferencia del LLM (con alta demanda de GPU y costo variable según el modelo), la base vectorial (con demanda de memoria y IOPS para la búsqueda), el pipeline de ingesta (con demanda de CPU y GPU para la generación de embeddings), y los servicios de soporte (autenticación, caching, orquestación). Escalar un componente sin entender los cuellos de botella de los demás produce arquitecturas que escalan parcialmente y fallan en el punto que no se optimizó.

La planificación de la escalabilidad comienza con la comprensión del perfil de carga. Los sistemas de IA tienen perfiles de carga distintos a los de los sistemas transaccionales convencionales. La inferencia de un LLM es CPU/GPU intensiva y tiene latencia variable según la longitud del contexto y la temperatura configurada. La búsqueda vectorial tiene alta demanda de memoria (el índice debe estar en RAM para búsquedas de baja latencia) y es sensible al número concurrente de consultas. El pipeline de ingesta es batch y tolerante a la latencia pero puede tener alta demanda de CPU/GPU para la generación de embeddings. Estas diferencias requieren estrategias de escalado diferenciadas por componente.

Los cuatro patrones de escalado que un arquitecto de IA debe dominar son:

Escalado horizontal: agregar más instancias de un servicio para distribuir la carga. Requiere que el servicio sea stateless o que el estado esté externalizado (en Redis, en una base de datos). El servicio de inferencia, el pipeline de recuperación y los agentes workers son candidatos naturales al escalado horizontal. La base vectorial puede escalar horizontalmente mediante sharding, pero requiere configuración cuidadosa para mantener la consistencia de búsqueda.

Escalado vertical: aumentar los recursos de una instancia existente (más CPU, más GPU, más memoria). Es apropiado cuando el servicio no puede fragmentarse de manera efectiva — por ejemplo, un modelo de reranking que necesita mantener toda su representación en memoria para funcionar con baja latencia. El escalado vertical tiene límites físicos y es más costoso por unidad de recurso que el escalado horizontal a gran escala.

Escalado automático (autoscaling): ajustar dinámicamente el número de instancias según la demanda en tiempo real. En servicios de inferencia, el autoscaling debe configurarse con métricas de cola de solicitudes y latencia, no solo con CPU — la latencia de respuesta es la señal más directa de que se necesita más capacidad. Kubernetes HPA (Horizontal Pod Autoscaler) con métricas personalizadas es el mecanismo estándar en entornos cloud.

Escalado serverless: para cargas de trabajo con demanda muy variable o intermitente, el modelo serverless — pagar solo por el tiempo de ejecución, sin capacidad reservada — puede ser más económico. Las funciones serverless son adecuadas para el pipeline de ingesta (ejecutado periódicamente, no continuamente) y para servicios de soporte ligeros. No son adecuadas para el servicio de inferencia principal si el cold start es inaceptable para la experiencia del usuario.

Las métricas que guían las decisiones de escalado en sistemas de IA incluyen:

- **Queue depth:** cuántas solicitudes están esperando ser procesadas. Un queue depth creciente es la señal más clara de insuficiencia de capacidad.
- **Time-to-first-token (TTFT):** en servicios de generación con streaming, el tiempo hasta que el usuario recibe el primer token es la métrica de experiencia más importante. La degradación del TTFT bajo carga indica necesidad de escalado del servicio de inferencia.
- **GPU utilization:** en infraestructura propia de inferencia, la utilización de GPU por debajo del 70-80% indica potencial de compactación; por encima del 90% indica necesidad de escalado.
- **Latencia de búsqueda vectorial:** el P95 y P99 de la búsqueda en la base vectorial deben mantenerse dentro de los SLO definidos bajo carga máxima esperada.

El principio rector de la escalabilidad en sistemas de IA es la sostenibilidad económica. Escalar sin controles de costo produce sistemas que soportan cualquier carga pero que no son financieramente viables a escala. La arquitectura correcta no es la que puede soportar la mayor carga posible, sino la que puede crecer de manera controlada manteniendo el equilibrio entre rendimiento, resiliencia y economía. Las secciones siguientes desarrollan cada una de las estrategias de escalado con herramientas concretas y criterios de decisión.

Módulo 4 – Capítulo 08 – Sección 02

Escalado Horizontal y Vertical

La elección entre escalado horizontal y vertical no es una preferencia tecnológica: es una decisión que depende de las características del componente a escalar, del perfil de demanda del sistema, y del presupuesto operativo. Para sistemas de IA, la mayoría de los componentes del pipeline tienen características que favorecen una estrategia específica, y la combinación de ambas estrategias en un mismo sistema es la norma, no la excepción.

Escalado horizontal del servicio de inferencia es el patrón predominante cuando se usan APIs de LLM comerciales (OpenAI, Anthropic, Google). En este caso, el "escalado" horizontal no significa agregar instancias del modelo propio sino distribuir las solicitudes entre múltiples claves de API o entre múltiples proveedores, usando estrategias de round-robin o de enrutamiento por carga. Cuando el equipo opera su propia infraestructura de inferencia — bien sea con modelos open source como Llama, Mistral o Qwen —, el escalado horizontal se implementa con servidores de inferencia especializados. Las opciones principales son:

- **vLLM:** servidor de inferencia open source que implementa PagedAttention para gestión eficiente de la KV cache, permitiendo alta concurrencia con la misma GPU. Soporta múltiples modelos y es la opción de referencia para inferencia de alta concurrencia en hardware propio o arrendado.
- **Text Generation Inference (TGI) de Hugging Face:** servidor de inferencia optimizado para modelos de la biblioteca Hugging Face, con soporte nativo para continuous batching (agrupa múltiples solicitudes para maximizar el uso de la GPU) y tensor parallelism para distribuir un modelo grande entre múltiples GPUs.
- **KServe:** plataforma de serving de modelos de IA sobre Kubernetes, agnóstica al framework de modelo. Proporciona escalado automático, A/B testing de modelos, y gestión del ciclo de vida. Es la opción preferida en organizaciones con infraestructura Kubernetes existente.
- **Ray Serve (ray.io):** sistema de serving escalable que funciona sobre el framework Ray de computación distribuida. Especialmente adecuado para pipelines de IA complejos

donde la lógica de serving requiere orquestación de múltiples modelos o componentes.

Escalado horizontal de la base vectorial requiere una comprensión de la arquitectura de la base vectorial específica. Pinecone gestiona el sharding automáticamente como servicio gestionado. Weaviate soporta escalado horizontal mediante sharding de colecciones entre múltiples nodos, con replicación para disponibilidad. Qdrant implementa sharding distribuido con rebalanceo automático. pgvector, al operar sobre PostgreSQL, escala a través de los mecanismos de PostgreSQL: réplicas de lectura para distribuir las consultas de búsqueda, y escalado vertical del nodo principal para las escrituras.

Escalado vertical es apropiado para componentes donde la distribución horizontal introduce overhead inaceptable. Los modelos de reranking — especialmente los cross-encoders de alta calidad como Cohere Rerank o BGE-Reranker-Large — se benefician del escalado vertical porque procesan la consulta y cada chunk de forma conjunta en la misma instancia, y fragmentar ese procesamiento introduce latencia. Una instancia más potente (más GPU memory, más VRAM) permite procesar batches más grandes y reducir la latencia total. Los modelos de embedding también pueden beneficiarse del escalado vertical cuando el volumen de indexación es alto: una GPU de mayor clase puede generar embeddings a mayor velocidad que dos GPUs de menor clase con el overhead de coordinación.

La combinación correcta para un sistema de IA productivo típico es:

- Servicio de inferencia: escalado horizontal con vLLM o TGI, autoscaling basado en queue depth y TTFT.
- Base vectorial: escalado horizontal mediante sharding gestionado, con réplicas de lectura para alta disponibilidad.
- Servicio de reranking: escalado vertical hasta el límite razonable, con escalado horizontal de backup para picos de demanda.
- Pipeline de ingesta: escalado horizontal con workers batch, sin restricciones de latencia.
- Servicios de soporte (cache, auth, orchestration): escalado horizontal stateless.

Nota del Arquitecto: El error más frecuente en el escalado de sistemas de IA es escalar el servicio de inferencia cuando el cuello de botella real está en la base vectorial o en el servicio de reranking. Antes de tomar cualquier decisión de escalado, siempre mida la latencia de cada etapa del pipeline bajo la carga que produce el problema. La respuesta "necesitamos más GPUs" casi siempre requiere validación:

¿es el GPU realmente el cuello de botella, o es la red entre servicios, o es la base vectorial que necesita más RAM para mantener el índice en caché?

La decisión de escalado horizontal vs. vertical no es permanente: a medida que el sistema crece, la estrategia óptima puede cambiar. Un sistema que comienza con una única instancia vertical potente puede migrar a una arquitectura horizontal a medida que la demanda crece y la paralelización se vuelve más eficiente que el escalado vertical adicional.

Módulo 4 – Capítulo 08 – Sección 03

Balanceo de Carga

El balanceo de carga en sistemas de IA tiene particularidades que lo distinguen del balanceo de carga en aplicaciones web convencionales. En una aplicación web, todas las solicitudes son estadísticamente equivalentes en términos de costo computacional: un request HTTP para obtener el perfil de un usuario cuesta aproximadamente lo mismo que otro. En un sistema de IA, la varianza del costo por solicitud puede ser de varios órdenes de magnitud: una consulta simple con contexto corto puede completarse en 0.3 segundos y consumir 500 tokens, mientras que una consulta compleja con contexto largo puede tardar 8 segundos y consumir 15.000 tokens. Un balanceador que distribuye solicitudes en round-robin sin considerar esa varianza producirá instancias desequilibradas: algunas procesando muchas solicitudes ligeras, otras saturadas por una sola solicitud pesada.

Balanceo con conciencia de carga es la estrategia que resuelve este problema. En lugar de distribuir solicitudes en round-robin, el balanceador dirige cada nueva solicitud a la instancia con menor carga actual. En servicios de inferencia, la "carga actual" puede medirse como el número de solicitudes en vuelo, el uso actual de GPU memory, o la longitud total del contexto siendo procesado. vLLM expone métricas de carga a través de su API que un balanceador inteligente puede consumir para tomar decisiones de routing más precisas.

Enrutamiento por características de la solicitud es una estrategia más sofisticada donde el balanceador dirige la solicitud no solo según la carga de las instancias sino según las características de la solicitud misma. Por ejemplo:

- Solicitudes que requieren el modelo más potente (por complejidad del razonamiento) se enrutan a instancias con GPUs de mayor clase.
- Solicitudes de inferencia simple (resúmenes, clasificaciones) se enrutan a instancias con modelos más ligeros y económicos.
- Solicitudes de usuarios premium (con SLO de latencia más estricto) se enrutan a un pool de instancias dedicado con capacidad reservada.
- Solicitudes de procesamiento batch (sin SLO estricto de latencia) se enrutan a instancias de menor costo o a capacidad spot de cloud.

Circuit breaker para APIs de LLM externas es un mecanismo de protección específico para sistemas que dependen de APIs de modelos de lenguaje de terceros. Las APIs de LLM pueden experimentar degradación o indisponibilidad parcial: respuestas lentas, errores de rate limit, o errores de servidor. Un circuit breaker monitorea la tasa de éxito de las llamadas a la API y, cuando detecta una degradación significativa, "abre el circuito" — deja de enviar solicitudes al proveedor degradado — y redirige el tráfico a un proveedor alternativo o a una respuesta de degradación controlada (como un mensaje que indica que el servicio está temporalmente indisponible). Las implementaciones comunes incluyen la biblioteca `resilience4j` para Java, `pybreaker` para Python, o el circuit breaker integrado en service meshes como Istio o Envoy.

Caché semántico como mecanismo de reducción de carga reduce el número de solicitudes que llegan al servicio de inferencia. Un caché semántico almacena las respuestas generadas para consultas anteriores y, cuando una nueva consulta es semánticamente similar (no necesariamente idéntica) a una consulta en caché, devuelve la respuesta almacenada sin invocar al LLM. Esto es especialmente efectivo para sistemas con distribución de consultas concentrada: cuando el 20% de las preguntas genera el 80% del tráfico. GPTCache y Langchain's SemanticCache son implementaciones de referencia. El umbral de similitud para considerar dos consultas "equivalentes" debe calibrarse cuidadosamente: demasiado permisivo produce respuestas incorrectas para consultas que son similares pero no equivalentes; demasiado estricto produce tasas de hit de caché demasiado bajas para justificar la complejidad.

Gestión de cola y prioridades permite que el sistema mantenga servicio de calidad para los usuarios prioritarios incluso bajo alta demanda. Una cola de prioridades asigna capacidad de procesamiento según el SLO del usuario (premium vs. estándar) o según la urgencia de la solicitud (sincrónica vs. batch). La separación de pools de recursos — instancias dedicadas para usuarios premium, instancias compartidas para tráfico estándar — garantiza que la alta demanda de usuarios estándar no impacte la latencia de usuarios con requisitos estrictos.

Sticky sessions para sistemas de agentes son necesarias cuando el estado del agente se mantiene en memoria de la instancia (en lugar de externalizarse completamente). En este caso, el balanceador debe dirigir todas las solicitudes de la misma sesión a la misma instancia del agente. Session affinity mediante cookies o headers de sesión es el mecanismo estándar. Sin embargo, la arquitectura más robusta es la que externaliza completamente el estado del agente (a Redis u otro almacenamiento externo), eliminando la necesidad de sticky sessions y permitiendo el escalado stateless.

El balanceo de carga bien diseñado es invisible para el usuario: el sistema responde con latencia consistente independientemente de qué instancia procesa cada solicitud. Cuando el balanceo falla, el síntoma más frecuente no es un error explícito sino una degradación de latencia que afecta solo a algunos usuarios — los que el balanceador está dirigiendo a instancias saturadas.

Módulo 4 – Capítulo 08 – Sección 04

Optimización de Costos

El costo operativo de un sistema de IA en producción puede ser sorprendentemente alto si no se gestiona activamente. Un sistema que realiza 100.000 consultas diarias con un contexto promedio de 10.000 tokens de entrada y 500 tokens de salida, usando un modelo de precio medio, puede generar una factura mensual de decenas de miles de dólares solo en costos de inferencia. A esto se suman los costos de la base vectorial, el almacenamiento, la red, los servicios de soporte y la infraestructura de GPU si el equipo opera sus propios modelos. La optimización de costos no es una consideración secundaria: es una dimensión de diseño que debe abordarse desde la arquitectura.

Selección del modelo por tipo de tarea es la palanca de optimización de mayor impacto. No todas las tareas en un sistema de IA requieren el modelo más potente. Una taxonomía de tareas por complejidad permite asignar cada tipo de consulta al modelo de menor costo que produce calidad aceptable:

- **Modelos de alta capacidad** (GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro): reservados para razonamiento complejo, síntesis de múltiples fuentes, generación de documentos largos, y cualquier tarea donde la calidad del output es crítica.
- **Modelos de capacidad media** (GPT-4o mini, Claude 3 Haiku, Gemini Flash): adecuados para la mayoría de las consultas de soporte, respuestas basadas en RAG con contexto bien recuperado, y síntesis de documentos cortos.
- **Modelos ligeros y especializados**: para tareas de clasificación, extracción de entidades, generación de embeddings, reranking, y cualquier tarea que pueda resolverse con un modelo fine-tuned más pequeño.

El enrutamiento inteligente entre modelos — usar el modelo ligero por defecto y escalar al modelo potente solo cuando la complejidad lo requiere — puede reducir el costo de inferencia entre un 40% y un 70% en sistemas con distribución mixta de consultas.

Continuous batching es la técnica más importante para maximizar la eficiencia de GPU en infraestructura propia. Un modelo LLM servido de forma ingenua procesa una solicitud a la

vez, con la GPU esperando mientras el cliente envía la siguiente solicitud. El continuous batching agrupa dinámicamente múltiples solicitudes en un solo forward pass de la GPU, maximizando su utilización. Servidores como vLLM y TGI implementan continuous batching de forma nativa, y pueden aumentar el throughput de GPU por un factor de 5-10x respecto al serving naive.

Caché de KV (Key-Value cache) es una técnica específica para sistemas con prefijos de prompt repetitivos. En un sistema de soporte donde el system prompt es idéntico para todas las consultas, la KV cache de ese prompt puede reutilizarse entre solicitudes, evitando el cómputo repetido del prefijo compartido. OpenAI ofrece Prompt Caching con descuento en tokens de prefijo repetidos. Anthropic tiene funcionalidad equivalente en su API. vLLM implementa prefix caching a nivel de infraestructura propia.

Caché semántico de respuestas (descrito en la sección de balanceo) es la técnica de mayor impacto en sistemas con distribución de consultas concentrada. En sistemas de soporte con preguntas frecuentes, un caché semántico bien calibrado puede eliminar el 30-50% de las llamadas al LLM, con un ahorro proporcional en costo de inferencia.

Procesamiento batch para el pipeline de ingesta reemplaza la indexación de documentos en tiempo real (costosa en GPU) por procesamiento batch programado (que puede ejecutarse en horarios de menor demanda o en hardware más económico). El pipeline de ingesta — extracción, limpieza, chunking, embedding — no tiene requisitos de latencia en tiempo real: puede ejecutarse en lotes cada hora, cada día o en cada actualización del repositorio de documentos. Ejecutar este procesamiento en instancias spot de AWS, preemptible VMs de GCP, o Spot VMs de Azure reduce el costo entre un 50% y un 80% respecto a instancias on-demand.

Right-sizing de la infraestructura es la práctica de ajustar el tamaño de cada instancia a la carga real que procesa. Muchos sistemas se despliegan con instancias sobredimensionadas "por si acaso" y operan con utilización media del 15-20%. Monitorear la utilización real durante varias semanas y redimensionar según el percentil P95 de la demanda — en lugar del pico máximo histórico — puede reducir costos de infraestructura significativamente sin impacto en la experiencia del usuario.

Nota del Arquitecto: El error más costoso que he visto en sistemas de IA productivos es usar el modelo más poderoso disponible para todas las consultas porque "es el que da mejores resultados en las demos". En producción, el 80% de las consultas son suficientemente simples como para que un modelo de categoría media

las resuelva con la misma calidad percibida por el usuario. Implementar el enrutamiento por complejidad en el primer mes de operación es la optimización de mayor retorno disponible.

La optimización de costos debe ser una práctica continua, no una iniciativa puntual. Los costos del sistema de IA deben monitorearse semanalmente, con alertas cuando el costo por consulta aumenta por encima de la línea base, y revisiones trimestrales de la arquitectura de costos a medida que los volúmenes de uso cambian y los precios de los proveedores evolucionan.

Módulo 4 – Capítulo 08 – Sección 05

Alta Disponibilidad

La alta disponibilidad de un sistema de IA tiene una particularidad que la distingue de la alta disponibilidad de sistemas convencionales: el componente más crítico — el modelo de lenguaje — frecuentemente está fuera del control operativo directo del equipo. Cuando el sistema depende de una API de LLM de terceros, la disponibilidad del sistema está parcialmente limitada por la disponibilidad del proveedor. Diseñar la alta disponibilidad de un sistema de IA significa tanto eliminar los puntos únicos de fallo propios como preparar estrategias de degradación controlada para cuando el proveedor del LLM no está disponible.

Redundancia multi-proveedor es la estrategia de alta disponibilidad más robusta para sistemas que usan APIs de LLM comerciales. El sistema configura al menos dos proveedores de LLM como primario y secundario. Bajo operación normal, todas las solicitudes van al proveedor primario. Cuando el circuit breaker detecta degradación del proveedor primario (latencia elevada, tasa de error elevada, o indisponibilidad total), el tráfico se redirige automáticamente al proveedor secundario. Esta estrategia requiere que el sistema esté abstraído del proveedor específico mediante una capa de interfaz uniforme: el mismo prompt debe producir respuestas aceptables en ambos proveedores. La estandarización de comportamiento entre modelos distintos es un desafío de ingeniería de prompts que debe evaluarse cuidadosamente.

Alta disponibilidad de la base vectorial se logra mediante replicación: múltiples réplicas del índice vectorial, sincronizadas, con capacidad de servir consultas de lectura desde cualquier réplica. Las principales bases vectoriales gestionadas (Pinecone, Weaviate Cloud) ofrecen alta disponibilidad como característica del servicio. Para bases vectoriales self-hosted, Qdrant soporta replicación distribuida nativa; Weaviate tiene un modo de alta disponibilidad con múltiples nodos; pgvector aprovecha la replicación streaming de PostgreSQL. El compromiso arquitectónico es entre la consistencia fuerte (más costosa en latencia de escritura) y la consistencia eventual (más eficiente pero con ventanas de inconsistencia entre réplicas).

Despliegues sin interrupción (zero-downtime deployments) son esenciales para sistemas productivos que operan continuamente. Las estrategias más utilizadas son:

- **Blue-green deployment:** dos entornos idénticos (azul y verde) donde solo uno está activo en cada momento. El nuevo código se despliega en el entorno inactivo, se valida, y el tráfico se cambia instantáneamente. Si hay problemas, el rollback es cambiar el tráfico de vuelta al entorno anterior.
- **Canary deployment:** el nuevo código se despliega gradualmente: primero al 1% del tráfico, luego al 10%, al 50%, y finalmente al 100%. Si las métricas de calidad o de error degradan durante el proceso, el despliegue se detiene automáticamente. Esta estrategia es especialmente valiosa para cambios de prompt o de modelo donde el impacto en la calidad no puede verificarse completamente en un entorno de staging.
- **Rolling deployment:** las instancias del servicio se actualizan de una en una (o en grupos pequeños), manteniendo la mayor parte del tráfico en la versión anterior durante el despliegue. Es la estrategia más sencilla pero requiere compatibilidad hacia atrás entre la versión anterior y la nueva.

Estrategias de degradación controlada son el mecanismo que permite al sistema mantenerse útil incluso cuando uno o más componentes están degradados o no disponibles. El diseño de degradación controlada debe ser explícito: para cada componente del sistema, definir qué funcionalidad puede ofrecerse si ese componente no está disponible. Si la base vectorial no está disponible, el sistema puede responder solo con el conocimiento paramétrico del LLM (con advertencia explícita al usuario). Si el LLM principal no está disponible, el sistema puede derivar a un modelo de menor calidad. Si ambos están degradados, el sistema puede derivar todas las consultas a un humano. La degradación controlada es preferible a la indisponibilidad total.

Recuperación de estado en sistemas de agentes requiere que el estado de las tareas en curso sea persistido de manera que pueda recuperarse si una instancia falla. El checkpointing periódico del estado del agente — serializado en Redis u otro almacenamiento con alta disponibilidad — permite reanudar tareas interrumpidas desde el último checkpoint sin pérdida de trabajo. LangGraph implementa checkpointing como primitiva nativa de su modelo de ejecución.

Tests de disponibilidad deben ejecutarse continuamente, no solo ante incidentes. Synthetic monitoring — solicitudes automáticas periódicas al sistema con respuestas esperadas conocidas — detecta degradaciones antes de que los usuarios reales las experimenten. Los synthetic monitors deben cubrir el flujo completo del sistema, no solo el

endpoint de la API, para detectar fallos en componentes internos como la base vectorial o el servicio de reranking.

La alta disponibilidad de un sistema de IA productivo no es una característica que se añade después de construir el sistema: es un conjunto de decisiones de diseño que se toman desde el inicio y que determinan la resiliencia del sistema ante fallos inevitables. Los sistemas de IA que atienden a usuarios en producción fallarán en algún momento; la pregunta es cuán rápido se recuperan y cuánta funcionalidad mantienen durante el fallo.

Módulo 4 – Capítulo 08 – Sección 06

Resumen

Este capítulo desarrolló la escalabilidad como la disciplina que determina si un sistema de IA puede crecer con el negocio sin convertirse en una carga operativa insostenible. Escalar un sistema de IA no es escalar un único servicio: es escalar un ecosistema de componentes interdependientes con perfiles de carga radicalmente distintos, y hacerlo de manera que el costo operativo crezca de forma proporcional al valor generado, no de forma exponencial.

Las estrategias de escalado horizontal y vertical deben aplicarse diferenciadamente por componente. El servicio de inferencia escala horizontalmente con servidores especializados como vLLM, TGI, KServe o Ray Serve, que maximizan la utilización de GPU mediante continuous batching. La base vectorial escala horizontalmente mediante sharding y replicación. Los modelos de reranking y los servicios con estado que no pueden fragmentarse eficientemente escalan verticalmente. El pipeline de ingesta escala horizontalmente con workers batch en instancias de bajo costo, aprovechando su tolerancia a la latencia.

El balanceo de carga en sistemas de IA va más allá del round-robin: el balanceo con conciencia de carga, el enrutamiento por características de la solicitud, el circuit breaker para APIs de LLM, y el caché semántico son mecanismos que distribuyen eficientemente una carga altamente heterogénea. La separación de pools de recursos para usuarios con distintos SLOs garantiza que la alta demanda de un segmento no degrada la experiencia de otro.

La optimización de costos es la dimensión de la escalabilidad más directamente conectada con la viabilidad del sistema. La selección del modelo por tipo de tarea — reservar los modelos poderosos para tareas que genuinamente los requieren — es la palanca de mayor impacto. El continuous batching, el prompt caching, el caché semántico de respuestas, y el procesamiento batch para el pipeline de ingesta son las optimizaciones técnicas más efectivas. La práctica de right-sizing basado en percentiles de demanda real, en lugar de capacidad de pico máximo, cierra el círculo.

La alta disponibilidad en sistemas de IA combina los mecanismos estándar — redundancia, despliegues sin interrupción, degradación controlada — con estrategias específicas del

dominio: redundancia multi-proveedor de LLM, checkpointing de estado de agentes, y synthetic monitoring del flujo completo del sistema. El resultado no es un sistema que nunca falla, sino un sistema que falla de manera controlada y se recupera rápidamente.

El principio que une todas las dimensiones de la escalabilidad es el equilibrio: la mejor arquitectura no es la que soporta la mayor carga posible, sino la que puede crecer de manera controlada, sostenible y económicamente viable. El arquitecto que diseña con ese principio en mente construye sistemas que el negocio puede operar con confianza a medida que crece.

El Capítulo 09 cierra el arco operativo del módulo con la disciplina que da coherencia a todo lo anterior: el gobierno de plataformas de IA. Mientras la observabilidad, la seguridad y la escalabilidad son disciplinas técnicas de operación, el gobierno es la disciplina organizacional que asegura que todos esos esfuerzos técnicos se traducen en capacidades institucionales sostenibles.

"La mejor arquitectura no es la que soporta la mayor carga, sino la que puede crecer de manera controlada, sostenible y económicamente viable."

— Principio de escalabilidad en plataformas de IA